

CSL351: Computer Networks

Assignment - 5

Name: Harshita Patidar
ID No: 12340920

Data Submission

As per the assignment requirements, the PCAP trace files generated during these experiments (Que 4 and Que 5) have been uploaded to Google Drive.

Google Drive Link:
[Click Here to Access PCAP Files](#)

Que 1: Simple Topology Creation

Objective

To create a simple network topology consisting of two nodes connected through a point-to-point link using ns-3 and visualize the topology using NetAnim.

Files Used

- que_1.cc
- que_1.xml (NetAnim output)

Topology Description

- The simulated network consists of Node 0 (Node1), Node 1 (Node2) and a **point-to-point wired link** between them..
- Network parameters: **Data Rate:** 5 Mbps, **Link Delay:** 2 ms, **IP Network:** 10.1.1.0/24

Code Explanation

Creating Nodes

```
1 NodeContainer nodes;  
2 nodes.Create (2);
```

This creates two nodes in the simulation representing Node1 and Node2.

Creating the Point-to-Point Link

```
1 PointToPointHelper pointToPoint;  
2 pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
3 pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));  
4 NetDeviceContainer devices = pointToPoint.Install (nodes);
```

Explanation:

- SetDeviceAttribute("DataRate", "5Mbps") → Sets link bandwidth to 5 Mbps.
- SetChannelAttribute("Delay", "2ms") → Sets propagation delay to 2 ms.
- Install(nodes) → Connects Node0 and Node1 with this link.

Installing Internet Stack

```
1 InternetStackHelper stack;  
2 stack.Install (nodes);
```

This installs the TCP/IP protocol stack on both nodes, enabling IP communication.

Assigning IP Addresses

```
1 Ipv4AddressHelper address;  
2 address.SetBase ("10.1.1.0", "255.255.255.0");  
3 Ipv4InterfaceContainer interfaces = address.Assign (devices);
```

Explanation: SetBase() defines network 10.1.1.0/24 and Assign() allocates IP addresses to nodes.

NetAnim Visualization

```
1 AnimationInterface anim ("task1_topology.xml");  
2 anim.SetConstantPosition(nodes.Get(0), 10.0, 10.0);  
3 anim.SetConstantPosition(nodes.Get(1), 30.0, 10.0);
```

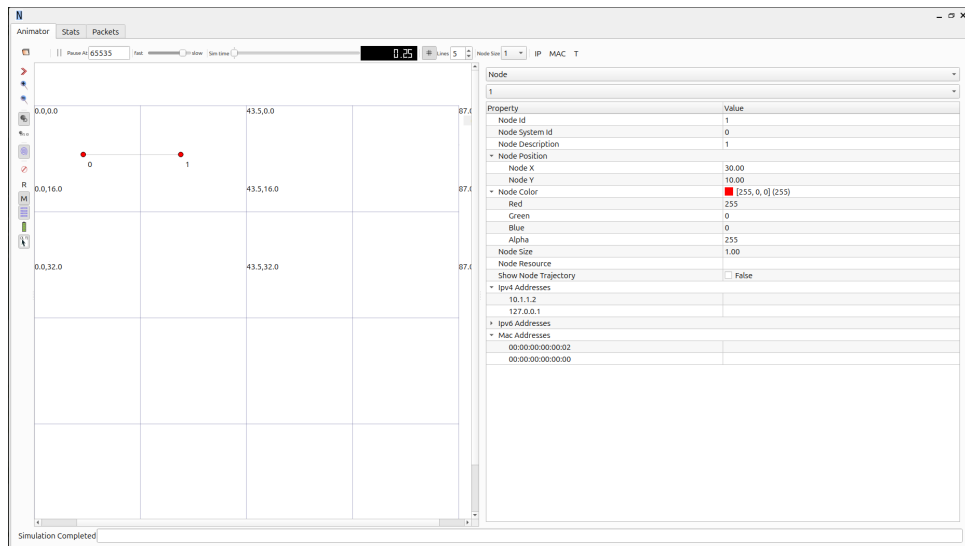
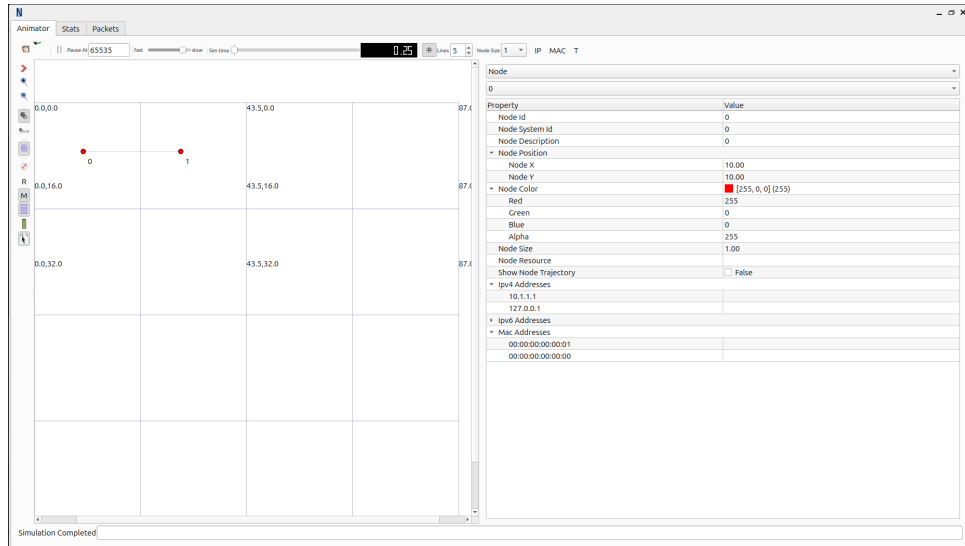
Explanation: Generates XML file for NetAnim.

Simulation Execution using below command:

```
./ns3 build  
./ns3 run que_1
```

This generates the file: que_1.xml

Output Visualization (NetAnim): Open que_1.xml in NetAnim



Inference

From this experiment, we observed:

- Nodes can be created in ns-3 using `NodeContainer`.
- Point-to-point links are configured using `PointToPointHelper`.
- Data rate and delay can be set via device and channel attributes.
- IP addresses are assigned using `Ipv4AddressHelper`.
- NetAnim helps visualize ns-3 topology.

Que 2: UDP Client-Server Setup

Objective

To establish UDP communication between two nodes by configuring a UDP client on Node 0 that sends packets to a UDP server on Node 1 at a fixed data rate.

Files Used

- `que_2.cc`
- `que_2.xml` (NetAnim output)

Topology Description

The physical topology (Nodes, Links, and IP Stack) is **identical** to Que 1.

- Extension: This task adds the **Application Layer** to the existing topology.
- Traffic Flow: Node 0 (Client) → Node 1 (Server).

Code Explanation

Since the Node and Link creation follows the same procedure as Que 1, we focus here on the Application setup.

UDP Server Setup (Node 1)

We used `UdpServerHelper` to open a listening port on the destination node.

```
1 uint16_t serverPort = 9;  
2 UdpServerHelper server (serverPort);  
3 ApplicationContainer serverApps = server.Install (nodes.Get (1));
```

This creates a sink for traffic on **Port 9** of Node 1.

UDP Client Setup & Fixed Data Rate

The `UdpClientHelper` was installed on Node 0. To achieve a fixed data rate, we did not use a simple "rate" attribute but instead configured the transmission physics directly:

```
1 UdpClientHelper client (interfaces.GetAddress (1), serverPort);  
2 client.SetAttribute ("Interval", TimeValue (Seconds (0.05)));  
3 client.SetAttribute ("PacketSize", UintegerValue (1024));
```

Calculation of Fixed Rate: By fixing the packet size and interval, the data rate becomes deterministic:

$$\text{Throughput} = \frac{\text{Packet Size}}{\text{Interval}} = \frac{1024 \text{ bytes}}{0.05 \text{ s}} \approx 163.84 \text{ Kbps}$$

Simulation Execution

The simulation was executed using `./ns3 run que_2`.

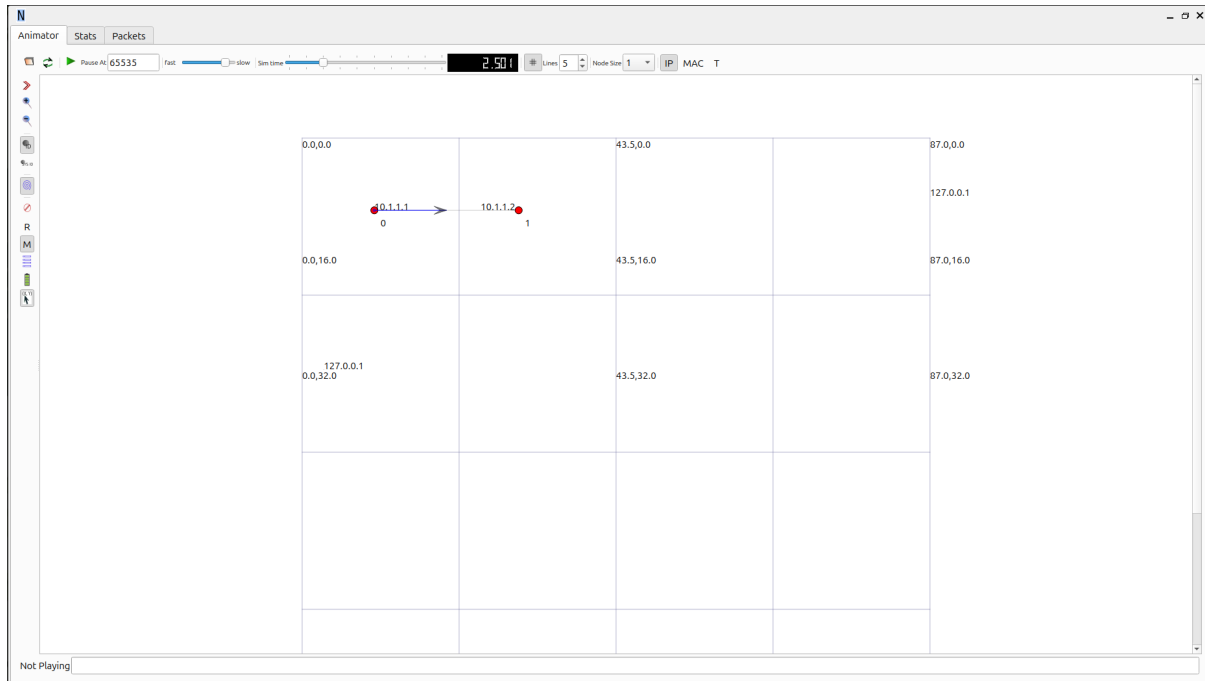
Observation: The Client (Node 0) successfully started sending packets at 2.0s, and the Server (Node 1) received them on Port 9.

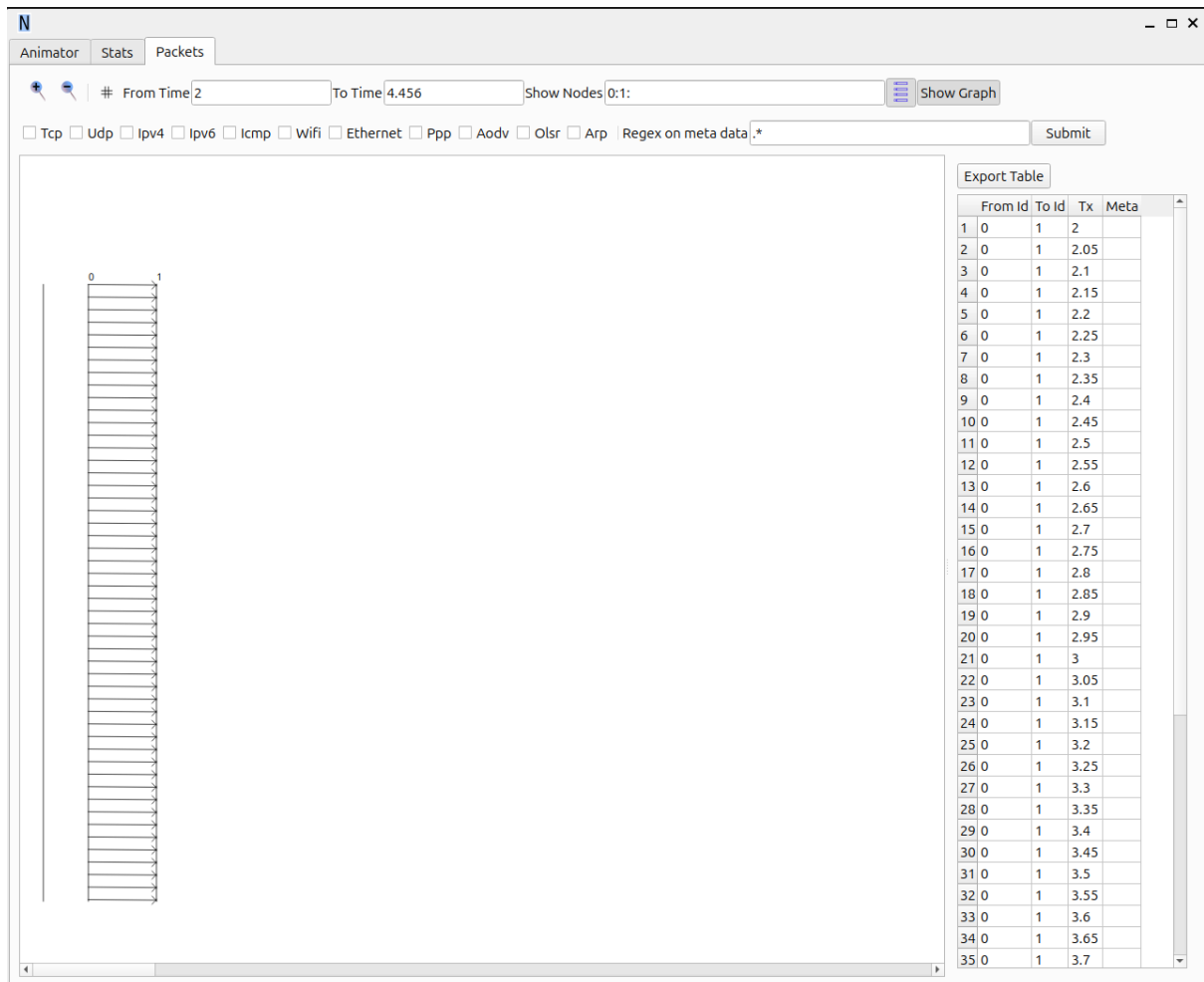
NetAnim Visualization

To visualize the packet flow for this specific UDP client-server scenario, we added the Animation Interface:

```
1 AnimationInterface anim ("que_2.xml");  
2 anim.SetConstantPosition(nodes.Get(0), 10.0, 10.0);  
3 anim.SetConstantPosition(nodes.Get(1), 30.0, 10.0);
```

This generates a trace file named `que_2.xml`, which captures the node positions and traffic flow.





Packet flow from Node0 to Node1 in Que2

Inference

- `UdpClientHelper` allows for precise traffic control via `Interval` and `PacketSize` attributes.
- The destination IP must be retrieved dynamically (using `interfaces.GetAddress(1)`) to ensure the client targets the correct node.

Que 3: Throughput Analysis

Objective

To measure the end-to-end throughput and packet transmission statistics of a point-to-point link while varying critical network parameters such as Bandwidth, Latency, Packet Size, and Transmission Interval.

Files Used

- `que_3.cc`

Topology Description

The underlying topology remains the standard 2-node point-to-point link. However, the attributes of this topology are dynamic and controlled via command-line arguments rather than being hardcoded.

Code Explanation

Dynamic Parameter Configuration

To facilitate testing multiple scenarios without recompiling, we used the `CommandLine` interface. This allows us to pass simulation parameters directly from the terminal.

```
1 CommandLine cmd;  
2 cmd.AddValue ("bandwidth", "Link bandwidth", bandwidth);  
3 cmd.AddValue ("latency", "Link latency", latency);  
4 cmd.AddValue ("maxPackets", "Max packets to send", maxPackets);  
5 cmd.AddValue ("interval", "Packet interval", interval);  
6 cmd.AddValue ("packetSize", "Packet size in bytes", packetSize);  
7 cmd.Parse (argc, argv);
```

Throughput Calculation using FlowMonitor

Instead of manually parsing PCAP files, we used the `FlowMonitor` module to extract metrics programmatically.

```
1 // Install FlowMonitor  
2 FlowMonitorHelper flowmon;  
3 Ptr<FlowMonitor> monitor = flowmon.InstallAll ();
```

Throughput Formula: After the simulation, we iterate through the flow statistics. The throughput is calculated as:

$$\text{Throughput (bps)} = \frac{\text{Received Bytes} \times 8}{\text{Time Last Rx} - \text{Time First Tx}}$$

Unit Formatting Logic: The code automatically adjusts the output units (Gbps, Mbps, Kbps) based on the magnitude of the result to match the assignment requirements:

```
1 if (throughputBps >= 1000000) {  
2     NS_LOG_UNCOND (" Throughput: " << throughputBps / 1000000 << " Mbps  
3     ");  
4 } else if (throughputBps >= 1000) {  
5     NS_LOG_UNCOND (" Throughput: " << throughputBps / 1000 << " Kbps");  
6 }
```

Simulation Execution

The simulation is executed with different arguments for each test case. **Example Command:**

```
./ns3 run "task3_optimized --bandwidth=5Kbps --latency=15ms  
--maxPackets=100 --interval=1s --packetSize=1024"
```

Output Observations

The following table summarizes the results obtained from the simulation runs:

Bandwidth	Latency	Max Packets	Interval	Packet Size	Throughput	Packets Tx
5 Kbps	15 ms	100	1 sec	1024	4.9871 Kbps	58
5 Kbps	10 ms	50	100 ms	512	4.97713 Kbps	50
5 Kbps	5 ms	Unlimited	10 ms	1024	4.98805 Kbps	5800
5 Mbps	15 ms	150	10 sec	2048	1.99223 Kbps	6
5 Mbps	10 ms	50	5 sec	256	495.615 bps	12
5 Mbps	5 ms	10	1 ms	256	1.57149 Mbps	10
5 Gbps	15 ms	Unlimited	1 ms	1024	8.41397 Mbps	58000
5 Gbps	20 ms	5	2 sec	256	1.41646 Kbps	5
5 Gbps	5 ms	100	5 ms	512	863.999 Kbps	100

Table 1: Network Performance Metrics for Varying Parameters

```

harshita-patidar@harshita-patidar:~/ns-allinone-3.44/ns-3.44
Finished executing the following commands:
/usr/bin/cmake --build /home/harshita-patidar/ns-allinone-3.44/ns-3.44/cmake-cache
-j 15
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Kbps --
latency=15ms --maxPackets=100 --interval=1s --packetSize=1024"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 58
  Rx Packets: 13
  Throughput: 4.9871 Kbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Kbps --
latency=10ms --maxPackets=50 --interval=100ms --packetSize=512"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 50
  Rx Packets: 13
  Throughput: 4.97713 Kbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Kbps --
latency=5ms --maxPackets=0 --interval=10ms --packetSize=1024"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 5800
  Rx Packets: 6
  Throughput: 4.98805 Kbps
-----

```



```
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Mbps --latency=15ms --maxPackets=150 --interval=10s --packetSize=2048"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 6
  Rx Packets: 6
  Throughput: 1.99223 Kbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Mbps --latency=10ms --maxPackets=50 --interval=5s --packetSize=256"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 12
  Rx Packets: 12
  Throughput: 495.615 bps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Mbps --latency=5ms --maxPackets=10 --interval=1ms --packetSize=256"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
  Tx Packets: 10
  Rx Packets: 10
  Throughput: 1.57149 Mbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Mbps --latency=5ms --maxPackets=10 --interval=1ms --packetSize=256"
```

```

harshita-patidar@harshita-patidar:~/ns-allinone-3.44/ns-3.44
Throughput: 1.57149 Mbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Gbps --
latency=15ms --maxPackets=0 --interval=1ms --packetSize=1024"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
Tx Packets: 58000
Rx Packets: 58000
Throughput: 8.41397 Mbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Gbps --
latency=20ms --maxPackets=5 --interval=2s --packetSize=256"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
Tx Packets: 5
Rx Packets: 5
Throughput: 1.41646 Kbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ ./ns3 run "que_3 --bandwidth=5Gbps --
latency=5ms --maxPackets=100 --interval=5ms --packetSize=512"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Configuring Simulation...
Starting Simulation (60s)...
Simulation Finished. Processing Stats...
-----
Flow 1 (10.1.1.1 -> 10.1.1.2)
Tx Packets: 100
Rx Packets: 100
Throughput: 863.999 Kbps
-----
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $

```

Inference

1. **Bandwidth Saturation:** In the 5 Kbps scenarios, the throughput consistently capped near ≈ 4.98 Kbps, indicating that the link bandwidth was the bottleneck.
2. **Impact of Interval:** When the transmission interval was very high (e.g., 5 seconds in Row 5), the throughput dropped drastically (495 bps) because the link remained idle for long periods.
3. **Packet Size Efficiency:** Larger packets (e.g., 1024 or 2048 bytes) generally yield better efficiency (goodput) compared to smaller packets, as the ratio of header overhead to payload is minimized.

Que 4: Adding Echo Server & Conflict Avoidance

Objective

To introduce a second application pair (UDP Echo Client and Server) to the existing two-node topology and configure it to run simultaneously with the standard UDP application without causing a port conflict.

Files Used

- `que_4.cc`
- `que4-0-0.pcap` (Node 0 Trace)
- `que4-1-0.pcap` (Node 1 Trace)

Topology Description

The topology remains the same (Node 0 connected to Node 1 via a 5 Mbps link). However, the application layer now supports two distinct traffic flows:

- **Flow 1:** Standard UDP Traffic (from Que 2).
- **Flow 2:** UDP Echo Traffic (New).

Code Explanation

Conflict Resolution (The Critical Step)

The primary challenge in this task is running two servers on the same node (Node 1). If both servers try to bind to the same port (e.g., Port 9), a conflict will occur, and the simulation will fail or behave unpredictably.

To resolve this, we configured the new UDP Echo application to use **Port 10**, ensuring it is distinct from the Standard UDP application on **Port 9**.

```
1 // App 1: Standard UDP Server on Port 9
2 uint16_t port1 = 9;
3 UdpServerHelper server1 (port1);
4
5 // App 2: UDP Echo Server on Port 10 (Avoids Conflict)
6 uint16_t port2 = 10;
7 UdpEchoServerHelper server2 (port2);
```

UDP Echo Client Configuration

The Echo Client was configured to send a small number of packets to verify connectivity without overwhelming the link.

```
1 UdpEchoClientHelper client2 (interfaces.GetAddress (1), port2);
2 client2.SetAttribute ("MaxPackets", UintegerValue (5)); // Send 5 echo
   packets
3 client2.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
4 client2.SetAttribute ("PacketSize", UintegerValue (1024));
```

Enabling PCAP Tracing

To verify that both flows are active and distinct, we enabled PCAP tracing for all point-to-point devices.

```
1 pointToPoint.EnablePcapAll ("que_4");
```

This generates `que4-0-0.pcap` and `que4-1-0.pcap` which can be analysed in Wireshark.

Simulation Execution

The simulation is executed using the command:

```
./ns3 run que_4
```

*Console Output:

Que 4: Running Two Applications.

1. Standard UDP Flow on Port 9
 2. UDP Echo Flow on Port 10 (No Conflict)
- PCAP traces enabled (prefix: que4)

Output Observation

- Two distinct application pairs start and run concurrently.
- The Standard UDP Client sends packets to Node 1:9.
- The Echo Client sends packets to Node 1:10 and receives replies.

Inference

From this experiment, we observed:

- **Port Multiplexing:** Multiple network applications can share the same physical node and IP address by using unique port numbers.
- **Conflict Avoidance:** Explicitly assigning different ports (9 and 10) prevents binding errors.
- **Verification:** Enabling PCAP tracing allows us to inspect packet headers and confirm that traffic is correctly directed to the intended ports.

Que 5: Multiple Application Instances

Objective

To configure and simulate multiple simultaneous application instances (Standard UDP and UDP Echo) over a single point-to-point link, demonstrating port-based multiplexing and analyzing the traffic using PCAP traces.

Files Used

- que_5.cc
- que_5.xml
- que5-0-0.pcap and que5-1-0.pcap

Topology Description

The physical topology remains a single point-to-point link between Node 0 and Node 1. However, the logical topology now supports **three distinct traffic flows** sharing the same 5 Mbps bandwidth:

- **Flow 1 (Port 9):** Standard UDP Traffic (High volume).
- **Flow 2 (Port 10):** UDP Echo Traffic (Request/Reply).
- **Flow 3 (Port 11):** Extra Standard UDP Traffic.

Code Explanation

Port-Based Multiplexing

To allow three different server applications to run on the same node (Node 1) without conflict, we assigned each a unique port number.

```
1 // App 1: Standard UDP on Port 9
2 UdpServerHelper server1 (9);
3
4 // App 2: UDP Echo on Port 10
5 UdpEchoServerHelper server2 (10);
6
7 // App 3: Extra UDP on Port 11
8 UdpServerHelper server3 (11);
```

Configuring "Unlimited" Traffic

The assignment requires unlimited transmission for specific durations. In ns-3, the UdpClient application interprets `MaxPackets = 0` as "send zero packets." Therefore, to simulate unlimited traffic, we set `MaxPackets` to a very large integer.

```
1 // Sets the limit to 100 million packets (effectively unlimited for 30s
  )
2 client1.SetAttribute ("MaxPackets", UintegerValue (100000000));
3 client1.SetAttribute ("Interval", TimeValue (Seconds (0.01)));
```

Enabling Visualization & Tracing

We enabled both NetAnim and PCAP tracing to verify the topology and traffic flows.

```
1 // Generates que5-0-0.pcap and que5-1-0.pcap
2 p2p.EnablePcapAll ("que5");
3
4 // Generates que_5.xml for NetAnim
5 AnimationInterface anim ("que_5.xml");
6 anim.SetConstantPosition (nodes.Get (0), 10.0, 10.0);
7 anim.SetConstantPosition (nodes.Get (1), 40.0, 10.0);
```

Simulation Execution

The simulation is executed using the command:

```
./ns3 run que_5
```

Generated Files:

- que_5.xml (NetAnim Trace)
- que5-0-0.pcap (Node 0 Trace)
- que5-1-0.pcap (Node 1 Trace)

Output Observation

- **Multiplexing:** Wireshark traces show packets arriving on ports 9, 10, and 11, confirming that multiple applications are running concurrently.
- **Traffic Volume:** The high number of packets in the trace file reflects the "unlimited" configuration.

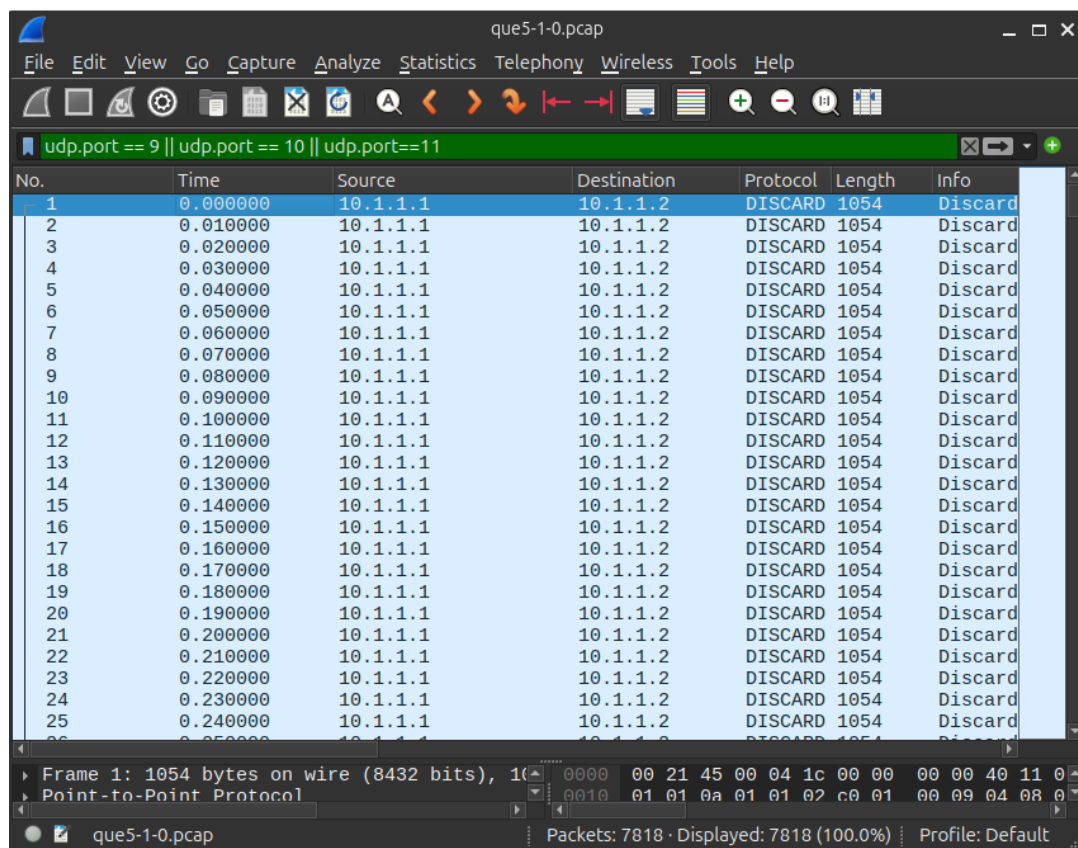


Figure 1: Wireshark Capture (Node 1 Trace) confirming successful delivery of multiple flows.

Inference

From this experiment, we observed:

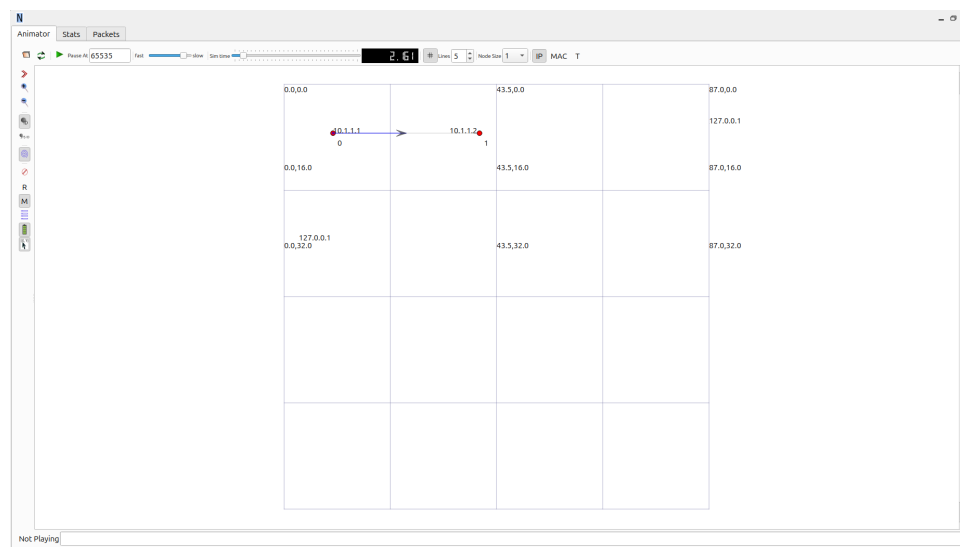
- **Resource Sharing:** Multiple network flows can share the same physical link bandwidth.
- **Port Usage:** Unique port numbers are essential for distinguishing between different applications on the same host.
- **Simulation Fidelity:** Using `EnablePcapAll` generates separate trace files for each interface, allowing detailed analysis of exactly what was sent (Node 0) versus what was received (Node 1).

Visualization and Analysis

Que 6: NetAnim Topology

To visualize the network layout, the simulation state was exported to an XML trace file using the `AnimationInterface` class in NS-3.

- **Generated File:** `que_5.xml`
- **Topology Description:** The visualization below confirms the placement of two nodes (Node 0 and Node 1) connected by a single point-to-point link.



NetAnim visualization of the 2-node topology used in Que 5.

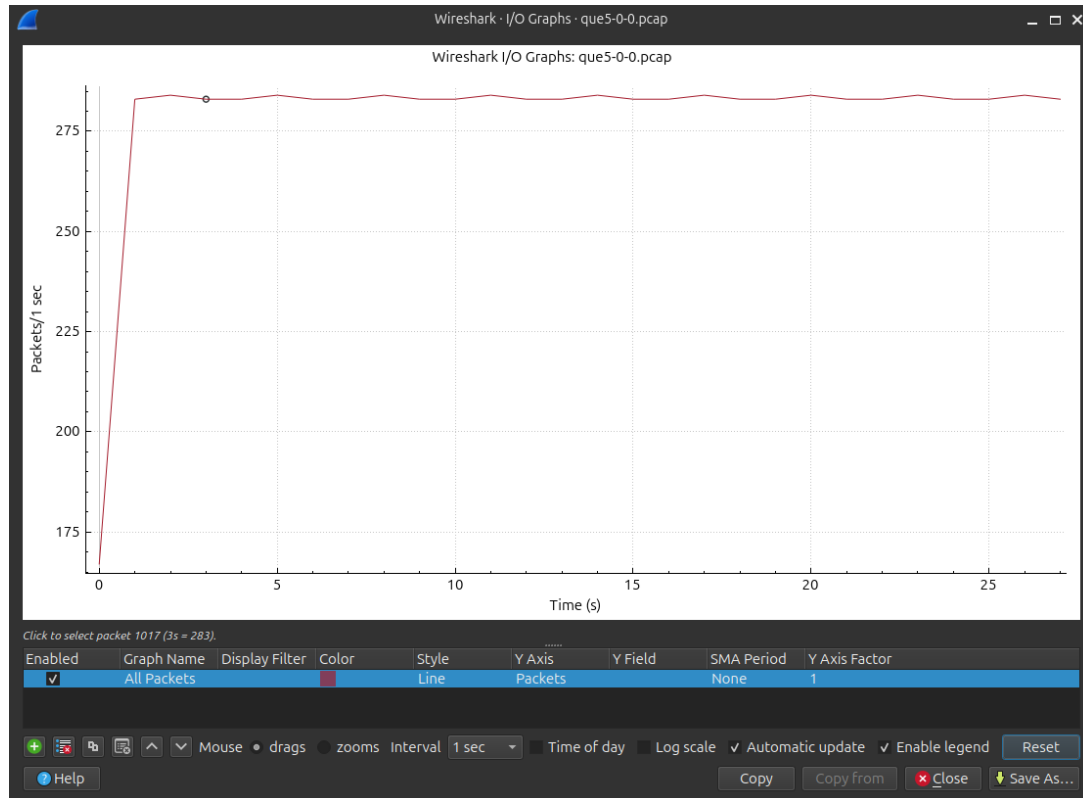
Que 7: Wireshark I/O Graphs

Objective

Use the pcap traces to generate an I/O graph using Wireshark for both nodes' interfaces to analyze throughput.

Graph 1: Node 0 Interface

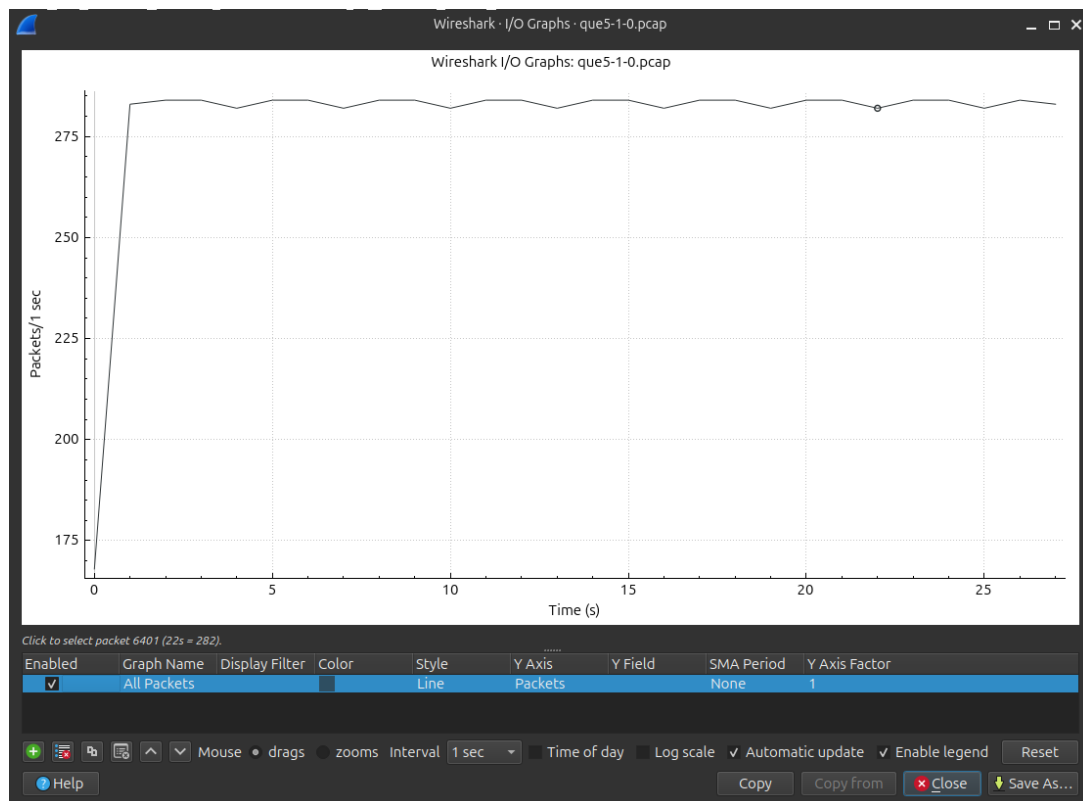
The graph below is generated from `que5-0-0.pcap`. Since Node 0 is the Client for all three applications, this graph primarily represents the **Outgoing Traffic (Tx)** load on the network.



Node 0 I/O Graph

Graph 2: Node 1 Interface

The graph below is generated from `que5-1-0.pcap`. Since Node 1 is the Server for all flows, this graph represents the **Incoming Traffic (Rx)** that was successfully delivered.



Node 1 I/O Graph

Inference

- **Symmetry:** The shape of the Node 0 (Tx) graph and Node 1 (Rx) graph is nearly identical. This confirms that the network is stable and not dropping significant amounts of traffic.
- **Throughput Level:** The graphs is showing a steady data rate corresponding to the 5 Mbps link capacity (saturated by the "Unlimited" traffic generation).